

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт цифрового развития
Кафедра инфокоммуникаций

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3
дисциплины «Объектно-ориентированное программирование»
Вариант 2

Выполнил:
Безруков Даниил Андреевич
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: перегрузка операторов в языке Python

Цель: приобретение навыков по перегрузке операторов при написании программ с помощью языка программирования Python версии 3.x.

Порядок выполнения работы:

Ссылка на репозиторий: https://github.com/pokachtononame/oop_lab_3

1. Дорабатываем пример из предыдущей работы, добавив перегрузки

```
class Rational:
    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)
        if b == 0:
            raise ValueError("Illegal value of the denominator")
        self.__numerator = a
        self.__denominator = b
        self.__reduce()

    # Сокращение дроби.
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)

        sign = 1
        if (self.__numerator > 0 and self.__denominator < 0) or (
            self.__numerator < 0 and self.__denominator > 0
        ):
            sign = -1
        a, b = abs(self.__numerator), abs(self.__denominator)
        c = gcd(a, b)
        self.__numerator = sign * (a // c)
        self.__denominator = b // c

    # Клонировать дробь.
    def __clone(self):
        return Rational(self.__numerator, self.__denominator)

    @property
    def numerator(self):
        return self.__numerator

    @numerator.setter
    def numerator(self, value):
        self.__numerator = int(value)
        self.__reduce()

    @property
    def denominator(self):
        return self.__denominator
```

```

@denominator.setter
def denominator(self, value):
    value = int(value)
    if value == 0:
        raise ValueError("Illegal value of the denominator")
    self.__denominator = value
    self.__reduce()

# Привести дробь к строке.
def __str__(self):
    return f"{self.__numerator} / {self.__denominator}"

def __repr__(self):
    return self.__str__()

# Привести дробь к вещественному значению.
def __float__(self):
    return self.__numerator / self.__denominator

# Привести дробь к логическому значению.
def __bool__(self):
    return self.__numerator != 0

# Сложение обыкновенных дробей.
def __iadd__(self, rhs): # +=
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator
        self.__numerator, self.__denominator = a, b
        self.__reduce()
        return self
    else:
        raise ValueError("Illegal type of the argument")

def __add__(self, rhs): # +
    return self.__clone().__iadd__(rhs)

# Вычитание обыкновенных дробей.
def __isub__(self, rhs): # -=
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator - self.denominator *
rhs.numerator
        b = self.denominator * rhs.denominator
        self.__numerator, self.__denominator = a, b
        self.__reduce()
        return self
    else:
        raise ValueError("Illegal type of the argument")

def __sub__(self, rhs): # -
    return self.__clone().__isub__(rhs)

# Умножение обыкновенных дробей.
def __imul__(self, rhs): # *=
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.numerator
        b = self.denominator * rhs.denominator
        self.__numerator, self.__denominator = a, b
        self.__reduce()
        return self
    else:
        raise ValueError("Illegal type of the argument")

```

```

def __mul__(self, rhs): # *
    return self.__clone().__imul__(rhs)

# Деление обыкновенных дробей.
def __itruediv__(self, rhs): # /=
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator
        b = self.denominator * rhs.numerator
        if b == 0:
            raise ValueError("Illegal value of the denominator")
        self.__numerator, self.__denominator = a, b
        self.__reduce()
        return self
    else:
        raise ValueError("Illegal type of the argument")

def __truediv__(self, rhs): # /
    return self.__clone().__itruediv__(rhs)

# Отношение обыкновенных дробей.
def __eq__(self, rhs): # ==
    if isinstance(rhs, Rational):
        return (self.numerator == rhs.numerator) and (
            self.denominator == rhs.denominator
        )
    else:
        return False

def __ne__(self, rhs): # !=
    if isinstance(rhs, Rational):
        return not self.__eq__(rhs)
    else:
        return False

def __gt__(self, rhs): # >
    if isinstance(rhs, Rational):
        return self.__float__() > rhs.__float__()
    else:
        return False

def __lt__(self, rhs): # <
    if isinstance(rhs, Rational):
        return self.__float__() < rhs.__float__()
    else:
        return False

def __ge__(self, rhs): # >=
    if isinstance(rhs, Rational):
        return not self.__lt__(rhs)
    else:
        return False

def __le__(self, rhs): # <=
    if isinstance(rhs, Rational):
        return not self.__gt__(rhs)
    else:
        return False

```

Листинг 1. Класс Rational с добавленными перегрузками

```

if __name__ == "__main__":
    r1 = Rational(3, 4)
    print(f"r1 = {r1}")

    r2 = Rational(5, 6)
    print(f"r2 = {r2}")

    print(f"r1 + r2 = {r1 + r2}")
    print(f"r1 - r2 = {r1 - r2}")
    print(f"r1 * r2 = {r1 * r2}")
    print(f"r1 / r2 = {r1 / r2}")
    print(f"r1 == r2: {r1 == r2}")
    print(f"r1 != r2: {r1 != r2}")
    print(f"r1 > r2: {r1 > r2}")
    print(f"r1 < r2: {r1 < r2}")
    print(f"r1 >= r2: {r1 >= r2}")
    print(f"r1 <= r2: {r1 <= r2}")

```

Листинг 2. Тестовый код для кода выше

```

r1 = 3 / 4
r2 = 5 / 6
r1 + r2 = 19 / 12
r1 - r2 = -1 / 12
r1 * r2 = 5 / 8
r1 / r2 = 9 / 10
r1 == r2: False
r1 != r2: True
r1 > r2: False
r1 < r2: True
r1 >= r2: False
r1 <= r2: True

```

Рисунок 1. Результат выполнения тестового кода

2. Выполняем индивидуальные задания.

Задание 1. Необходимо выполнить индивидуальное задание 1 лабораторной работы 4.1, но максимально задействовав имеющиеся в Python средства перегрузки операторов.

Решение:

```

class Pair:
    def __init__(self, a=0.0, b=0.0):
        if isinstance(a, (float, int)) and isinstance(b, (float, int)):
            self.first = float(a)
            self.second = float(b)
        else:
            raise ValueError('Некорректный тип аргументов')

    # --- Строковое представление ---

    def __str__(self):
        return f"({self.first}, {self.second})"

    def __repr__(self):

```

```

        return f"Pair({self.first}, {self.second})"

# --- Арифметические операции ---

# Сложение (+)
def __add__(self, other):
    if isinstance(other, Pair):
        return Pair(self.first + other.first, self.second + other.second)
    if isinstance(other, (int, float)):
        return Pair(self.first + float(other), self.second +
float(other))
    raise ValueError('Некорректный тип аргумента')

def __radd__(self, other):
    return self.__add__(other)

# Вычитание (-)
def __sub__(self, other):
    if isinstance(other, Pair):
        return Pair(self.first - other.first, self.second - other.second)
    if isinstance(other, (int, float)):
        return Pair(self.first - float(other), self.second -
float(other))
    raise ValueError('Некорректный тип аргумента')

def __rsub__(self, other):
    if isinstance(other, (int, float)):
        return Pair(float(other) - self.first, float(other) -
self.second)
    raise ValueError('Некорректный тип аргумента')

# Умножение (*)
def __mul__(self, other):
    if isinstance(other, Pair):
        return Pair(self.first * other.first, self.second * other.second)
    if isinstance(other, (int, float)):
        return Pair(self.first * float(other), self.second *
float(other))
    raise ValueError('Некорректный тип аргумента')

def __rmul__(self, other):
    return self.__mul__(other)

# Деление (/)
def __truediv__(self, other):
    if isinstance(other, Pair):
        if other.first == 0.0 or other.second == 0.0:
            raise ZeroDivisionError("Деление на ноль")
        return Pair(self.first / other.first, self.second / other.second)
    if isinstance(other, (int, float)):
        if float(other) == 0.0:
            raise ZeroDivisionError("Деление на ноль")
        return Pair(self.first / float(other), self.second /
float(other))
    raise ValueError('Некорректный тип аргумента')

def __rtruediv__(self, other):
    if self.first == 0 or self.second == 0:
        raise ZeroDivisionError("Деление на ноль")
    if isinstance(other, (int, float)):
        return Pair(float(other) / self.first, float(other) /
self.second)
    raise ValueError('Некорректный тип аргумента')

```

```

# Возведение в степень (**)
def __pow__(self, other):
    if isinstance(other, Pair):
        return Pair(self.first ** other.first, self.second **
other.second)
    if isinstance(other, (int, float)):
        return Pair(self.first ** float(other), self.second **
float(other))
    raise ValueError('Некорректный тип аргумента')

# --- Унарные операции ---

def __neg__(self):
    return Pair(-self.first, -self.second)

def __abs__(self):
    return Pair(abs(self.first), abs(self.second))

# --- Операции сравнения ---

def __eq__(self, other):
    if not isinstance(other, Pair):
        return False
    return self.first == other.first and self.second == other.second

def __lt__(self, other):
    if not isinstance(other, Pair):
        raise ValueError('Некорректный тип аргумента')
    return (self.first, self.second) < (other.first, other.second)

def __le__(self, other):
    if not isinstance(other, Pair):
        raise ValueError('Некорректный тип аргумента')
    return (self.first, self.second) <= (other.first, other.second)

# --- Индексирование ---

def __getitem__(self, index):
    if index == 0: return self.first
    if index == 1: return self.second
    raise ValueError("Индекс может быть только 0 или 1")

def __setitem__(self, index, value):
    if not isinstance(value, (int, float)):
        raise ValueError("Значение должно быть числом")
    if index == 0:
        self.first = float(value)
    elif index == 1:
        self.second = float(value)
    else:
        raise ValueError("Индекс может быть только 0 или 1")

```

Листинг 3. Перегрузки, добавленные в класс Pair из предыдущей работы.

В коде приведены перегрузки, реализующие операции сложения, вычитания, умножения, деления в вариантах на число и на объект Pair, операции сравнения (больше, меньше, равно), возведения в степень, преобразования в строку, унарные операции и индексирование.

```

if __name__ == "__main__":
    p1 = Pair(4, 8)
    p2 = Pair(2, 2)

    print(p1 + p2)
    print(p1 / p2)

    print(p1 + 10)
    print(10 + p1)
    print(20 - p1)

    print(p1 < p2)
    print(p2 <= p1)

    print(p1[0])

```

Листинг 4. Тестовый код для задания 1.

```

(6.0, 10.0)
(2.0, 4.0)
(14.0, 18.0)
(14.0, 18.0)
(16.0, 12.0)
False
True
4.0

```

Рисунок 2. Результат выполнения тестового кода для класса Pair

Задание 2. Необходимо дополнительно к требуемым в заданиях операциям перегрузить операцию индексирования []. Максимально возможный размер списка задать константой. В отдельном поле «size» должно храниться максимальное для данного объекта количество элементов списка; реализовать метод «size()», возвращающий установленную длину. Если количество элементов списка изменяется во время работы, определить в классе поле «count». Первоначальные значения «size» и «count» устанавливаются конструктором.

Создать класс Decimal для работы с беззнаковыми целыми десятичными числами, используя для представления список из 100 элементов типа int, каждый из которых является десятичной цифрой. Младшая цифра

имеет меньший индекс (единицы – в нулевом элементе списка). Реальный размер списка задаётся как аргумент конструктора инициализации. Реализовать арифметические операции, аналогичные встроенным для целых и операции сравнения.

Решение:

```
class Decimal:
    MAX_SIZE = 100

    def __init__(self, size, value=0):
        if not (1 <= size <= Decimal.MAX_SIZE):
            raise ValueError(f"Размер списка должен быть от 1 до {Decimal.MAX_SIZE}")

        self._size = size
        self.digits = [0] * Decimal.MAX_SIZE

        val_str = str(value)[::-1]
        self.count = max(1, min(len(val_str), size))

        for i in range(min(len(val_str), size)):
            self.digits[i] = int(val_str[i])

    def size(self):
        return self._size

    def __getitem__(self, index):
        if not (0 <= index < self._size):
            raise IndexError("Индекс вышел за пределы size")
        return self.digits[index]

    def __setitem__(self, index, value):
        if not (0 <= index < self._size):
            raise IndexError("Индекс вышел за пределы size")
        if not (0 <= value <= 9):
            raise ValueError("Должна быть цифра от 0 до 9")

        self.digits[index] = value

        if index >= self.count and value > 0:
            self.count = index + 1
        elif index == self.count - 1 and value == 0:
            while self.count > 1 and self.digits[self.count - 1] == 0:
                self.count -= 1

    def __add__(self, other):
        if not isinstance(other, Decimal):
            raise ValueError('Некорректный тип аргумента')

        len_digits = max(self._size, other._size)
        res = Decimal(len_digits, 0)
        carry = 0

        for i in range(len_digits):
            full_sum = self.digits[i] + other.digits[i] + carry
            partial_sum = full_sum % 10
            carry = full_sum // 10
```

```

        res.digits[i] = partial_sum

    res.count = len_digits
    if carry > 0:
        if res.count >= Decimal.MAX_SIZE:
            raise OverflowError("Переполнение: число превышает 100
знаков")
        res.digits[res.count] = carry
        res.count += 1

    if res.count > res._size:
        res._size = res.count

    while res.count > 1 and res.digits[res.count - 1] == 0:
        res.count -= 1

    return res

    def __sub__(self, other):
        if not isinstance(other, Decimal): raise ValueError('Некорректный тип
аргумента')

        if self < other:
            raise ValueError('Результат вычитания не может быть
отрицательным')
        len_digits = max(self._size, other._size)
        res = Decimal(len_digits, 0)
        borrow = 0

        for i in range(len_digits):
            diff = self.digits[i] - other.digits[i] - borrow
            if diff < 0:
                diff += 10
                borrow = 1
            else:
                borrow = 0
            res.digits[i] = diff

        res.count = len_digits

        while res.count > 1 and res.digits[res.count - 1] == 0:
            res.count -= 1
        return res

    def __mul__(self, other):
        if not isinstance(other, Decimal): raise ValueError('Некорректный тип
аргумента')

        res = Decimal(Decimal.MAX_SIZE, 0)

        counter = Decimal(Decimal.MAX_SIZE, 0)
        for i in range(other.count):
            counter.digits[i] = other.digits[i]
            counter.count = other.count

        one = Decimal(Decimal.MAX_SIZE, 1)
        zero = Decimal(Decimal.MAX_SIZE, 0)

        while not (counter == zero):
            res = res + self
            counter = counter - one

        if res.count > Decimal.MAX_SIZE:
            raise OverflowError("Переполнение: результат умножения

```

```

превышает 100 знаков")

    res._size = res.count
    return res

def __eq__(self, other):
    if not isinstance(other, Decimal):
        return False

    if self.count != other.count:
        return False

    for i in range(self.count):
        if self.digits[i] != other.digits[i]:
            return False
    return True

def __lt__(self, other):
    if not isinstance(other, Decimal): raise ValueError('Некорректный тип аргумента')

    if self.count != other.count:
        return self.count < other.count

    for i in range(self.count - 1, -1, -1):
        if self.digits[i] != other.digits[i]:
            return self.digits[i] < other.digits[i]

    return False

def __le__(self, other):
    return self < other or self == other

def __str__(self):
    active_digits = self.digits[:self.count]
    return "".join(map(str, active_digits[::-1]))

```

Листинг 5. Класс Decimal

```

if __name__ == "__main__":
    a = Decimal(5, 99999)
    b = Decimal(5, 12345)
    print(a + b)
    print(a - b)
    print(a * b)
    b[0] = 6
    print(a + b)
    print(a - b)
    print(a * b)
    print(a < b)
    print(b <= a)

```

Листинг 6. Тестовый код для класса Decimal

```
112344
87654
1234487655
112345
87653
1234587654
False
True
```

Рисунок 3. Результат выполнения тестового кода

3. Фиксируем изменения и отправляем их в удалённый репозиторий

Ответы на контрольные вопросы:

1. Какие средства существуют в Python для перегрузки операций?

Перегрузка операторов — это один из ключевых способов реализации полиморфизма в объектно-ориентированном программировании, позволяющий задать собственную логику работы стандартных операций для пользовательских классов.

В языке Python для этого существуют специальные встроенные методы (так называемые магические методы). Как правило, они не вызываются напрямую, а автоматически активируются интерпретатором при использовании соответствующих операторов или встроенных функций.

Пример: Метод `__init__` перегружает конструктор класса. Конструктор отвечает за создание экземпляра класса и определение его начальных свойств.

2. Какие существуют методы для перегрузки арифметических операций и операций отношения в языке Python?

Для перегрузки арифметических операций используются следующие методы:

`__add__(self, other)` — для сложения (+).

`__sub__(self, other)` — для вычитания (-).

`__mul__(self, other)` — для умножения (*).

`__truediv__(self, other)` — для деления (/).

Для перегрузки операций сравнения используются методы:

`__eq__(self, other)` — для проверки равенства (==).

`__ne__(self, other)` — для проверки неравенства (!=).

`__lt__(self, other)` — для проверки «меньше чем» (<).

`__le__(self, other)` — для проверки «меньше или равно» (<=).

`__gt__(self, other)` — для проверки «больше чем» (>).

`__ge__(self, other)` — для проверки «больше или равно» (>=).

3. В каких случаях будут вызваны следующие методы: `__add__`, `__iadd__` и `__radd__`? Приведите примеры.

Данные методы перегружают операцию сложения для различных контекстов её использования:

`__add__` (Обычное сложение): Этот метод вызывается, когда объект класса стоит слева в стандартной операции сложения.

Пример: Выражение `a + b` интерпретатор преобразует в вызов `a.__add__(b)`.

`__iadd__` (Составное присваивание / In-place): Этот метод вызывается для операции `+=`, которая выполняет сложение с одновременным присваиванием результата и модифицирует объект «на месте».

Пример: Выражение `a += b` интерпретатор преобразует в вызов `a.__iadd__(b)`.

`__radd__` (Правостороннее / Отраженное сложение): Этот метод вызывается, если первый (левый) операнд не поддерживает сложение с данным классом. В этом случае Python пытается вызвать метод у второго (правого) операнда.

Пример: Выражение `b + a` вызовет метод `a.__radd__(b)`, если объект `b` не имеет собственного метода `__add__` для работы с объектом `a`.

4. Для каких целей предназначен метод `__new__`? Чем он отличается от метода `__init__`?

Главное отличие между этими методами заключается в разделении этапов создания объекта и его наполнения данными:

Метод `__new__` предназначен для непосредственного создания нового экземпляра класса. Он вызывается самым первым и отвечает за низкоуровневое выделение памяти под новый объект. Метод `__new__` обязан вернуть созданный экземпляр класса.

Метод `__init__` вызывается уже после `__new__`. Он не создает объект, а занимается его инициализацией, то есть настраивает начальное состояние уже созданного экземпляра (заполняет его поля значениями).

5. Чем отличаются методы `__str__` и `__repr__`?

Оба метода формируют строковое представление объекта, но служат для разных целей:

Метод `__str__` предназначен для возвращения «читаемого», понятного человеку текстового представления объекта. Именно оно используется по умолчанию для вывода информации пользователю, например, при вызове встроенной функции `print()` или `str()`.

Метод `__repr__` возвращает более «официальное» и детальное строковое представление объекта. Оно должно быть однозначным и, по

возможности, содержать валидный код, позволяющий воссоздать этот объект при передаче строки в функцию `eval()`.

Таким образом, `__str__` ориентирован на удобное отображение текста для пользователя, а `__repr__` — на отладку и разработку.

Вывод: в ходе выполнения работы были приобретены навыки по работе с перегрузками операторов при написании программ с помощью языка программирования Python версии 3.x. На практике данные навыки были успешно применены при разработке пользовательских классов для работы с математическими парами чисел и реализации поразрядной арифметики беззнаковых десятичных чисел.